



Secure Secrets with Secrets Manager



Adeem Akhtar

```
Code Blame 38 lines (28 loc) · 1.08 KB
1
2 import boto3
3 from botocore.exceptions import ClientError
4 import json
5
6 def get_secret():
7
8     secret_name = "aws-access-key"
9     region_name = "us-west-1"
10
11     # Create a Secrets Manager client
12     session = boto3.session.Session()
13     client = session.client(
14         service_name='secretsmanager',
15         region_name=region_name
16     )
17
18     try:
19         get_secret_value_response = client.get_secret_value(
20             SecretId=secret_name
21         )
22     except ClientError as e:
23         # For a list of exceptions thrown, see
24         # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
25         raise e
26
27     secret = get_secret_value_response['SecretString']
28
29     return json.loads(secret)
30
31 # Retrieve credentials from Secrets Manager
32 credentials = get_secret()
33
34 # Extract the values; if AWS_REGION isn't in the secret, use the region from the session
35 AWS_ACCESS_KEY_ID = credentials.get("AWS_ACCESS_KEY_ID")
36 AWS_SECRET_ACCESS_KEY = credentials.get("AWS_SECRET_ACCESS_KEY")
37 AWS_REGION = credentials.get("AWS_REGION", boto3.session.Session().region_name or "us-east-2")
```



Introducing Today's Project!

In this project, I will demonstrate the following: 1. Identify how a web app is insecurely storing credentials. 2. See how GitHub's secret scanning feature can block insecure code from being pushed to a repository. 3. Update the web app to use AWS Secrets Manager to store and retrieve credentials securely. 4. Verify that secured web app code can be made public without exposing sensitive credentials. I'm doing this project to learn cloud security.

Tools and concepts

Services I used were AWS Secrets Manager, AWS IAM, and GitHub. Key concepts I learnt include securing credentials.

Project reflection

This project took me approximately 1.5 hours. The most challenging part was git rebase. It was most rewarding to learn how to rebase the branch commit.

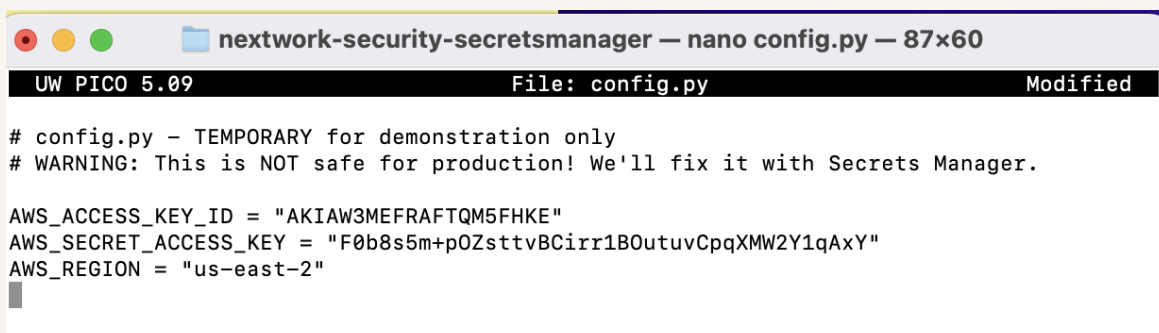
I chose to do this project today because I just finished the theory of AWS Secret Manager.



Hardcoding credentials

In this project, a sample web app is exposing AWS access key and AWS secret access key. It is unsafe to hardcode credentials because anyone who can see the hardcoded credentials can access the AWS account. Once someone else gets access to the credentials, they can use them to access your AWS account, delete resources, steal data, and cause damage.

I've set up the initial configuration with the access key and secret access key. These credentials are just examples, as we are experimenting with the project.



```
nextwork-security-secretsmanager — nano config.py — 87x60
UW PICO 5.09 File: config.py Modified
# config.py - TEMPORARY for demonstration only
# WARNING: This is NOT safe for production! We'll fix it with Secrets Manager.

AWS_ACCESS_KEY_ID = "AKIAW3MEFRAFTQM5FHKE"
AWS_SECRET_ACCESS_KEY = "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"
AWS_REGION = "us-east-2"
```

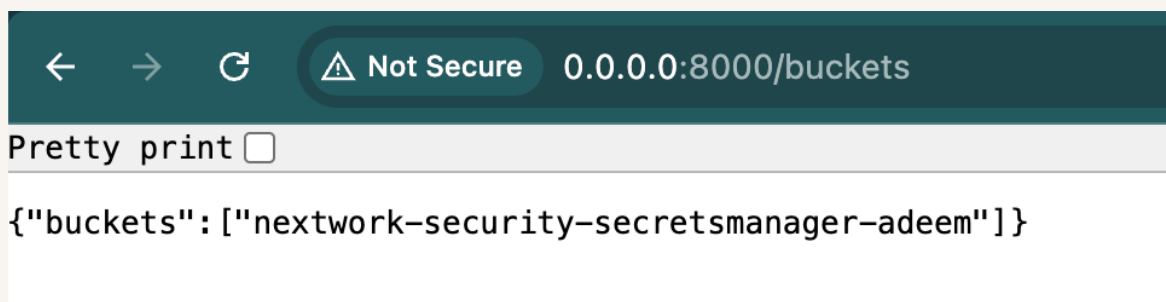


Using my own AWS credentials

As an extension for this project, I also decided to install the required libraries and set up my virtual environment. I installed Python and activated the virtual environment.

When I first ran the app, I ran into an error because there was no AWS access key and AWS secret access key.

To resolve the 'InvalidAccessKeyId' error, I updated AWS access, AWS Secret Access key, along with the region in the config.py file.





Pushing Insecure Code to GitHub

Once I updated the web app code with credentials, I forked the repository so I could have the same repository in my own GitHub account. A fork is different from a clone; forking is making a copy of it in your GitHub account online. If your forked repository is public, anyone can see it! Whereas cloning a repository, you're creating a local, offline copy. By default, no one can see it since it's stored on your computer.

To connect my local repository to the forked repository, I updated my config.py file. Then I used git add and git commit to stage. Finally, git push uploads the file to the GitHub repo.

GitHub blocked my push because my config.py file contained my AWS secret keys. This is a good security feature because GitHub does not let you push the code to a repo with the secret keys in it to protect the credentials.

```
((env) adeemakhtar@Mac nextwork-security-secretsmanager % git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 799 bytes | 799.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote: error: GH013: Repository rule violations found for refs/heads/main.
remote:
remote: - GITHUB PUSH PROTECTION
remote:
remote:   Resolve the following violations before pushing again
remote:
remote:   - Push cannot contain secrets
remote:
```



Secrets Manager

Secrets Manager is a service that helps you securely store and manage secrets, such as database credentials, API keys, and other sensitive information. Think of it as a digital vault for your secrets. I'm using it to store my access and secret access keys. Other common use cases include 'Database credentials', 'API Keys', 'OAuth tokens', etc.

Another feature in Secrets Manager is secret rotation, which means it's useful in situations where high-risk credentials like database passwords, privileged API keys, and service account credentials are used, and best security practices are to be followed.

Secrets Manager provides sample code in various languages, like Java, Python, Go, Rust, etc.

Sample code

Use these code samples to retrieve the secret in your application.

Java

JavaScript

C#

Python3

Ruby

Go

Rust

PHP

```
1 // Use this code snippet in your app.
2 // If you need more information about configurations or implementing the sample
3 // code, visit the AWS docs:
4 // https://docs.aws.amazon.com/sdk-for-java/latest/developer-guide/home.html
5
6 // Make sure to import the following packages in your code
7 // import software.amazon.awssdk.regions.Region;
8 // import software.amazon.awssdk.services.secretsmanager.SecretsManagerClient;
9 // import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueRequest;
10 // import software.amazon.awssdk.services.secretsmanager.model.GetSecretValueResponse;
11
12 public static void getSecret() {
13
14     String secretName = "aws-access-key";
15     Region region = Region.of("us-west-1");
```

Java

Line 1, Column 1

Errors: 0

Warnings: 0

[Download AWS SDK for Java](#)

Cancel

Previous

Store

Mobile App

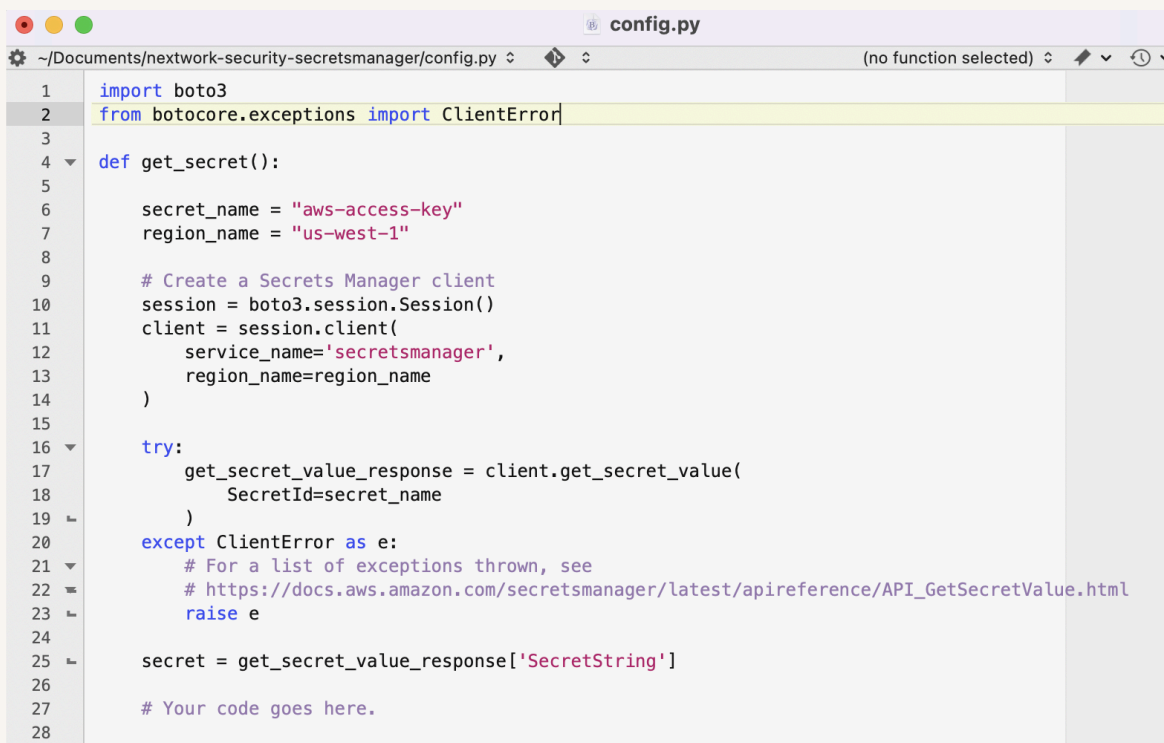
© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)



Updating the web app code

I updated the config.py file to retrieve defining get_secret() function. The get_secret() function will retrieve the secret keys from the secret manager.

I also added code to config.py to extract AWS_ACCESS_KEY, AWS_SECRET_ACCESS_KEY, and AWS_REGION from the AWS Secret Manager.



```
1 import boto3
2 from botocore.exceptions import ClientError
3
4 def get_secret():
5
6     secret_name = "aws-access-key"
7     region_name = "us-west-1"
8
9     # Create a Secrets Manager client
10    session = boto3.session.Session()
11    client = session.client(
12        service_name='secretsmanager',
13        region_name=region_name
14    )
15
16    try:
17        get_secret_value_response = client.get_secret_value(
18            SecretId=secret_name
19        )
20    except ClientError as e:
21        # For a list of exceptions thrown, see
22        # https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
23        raise e
24
25    secret = get_secret_value_response['SecretString']
26
27    # Your code goes here.
28
```



Rebasing the repository

Git rebasing is taking the pointer to the required commit. I used it to point my pointer to the root.

A merge conflict occurred during rebasing because When Git detects conflicting changes in a file, it adds special markers to show you exactly where the conflicts are: <<<<<< HEAD marks the beginning of your current version ===== separates the two conflicting versions >>>>>> feature-branch marks the end of the incoming changes. I resolved the merge conflict by deleting the entire section between <<<<<< HEAD and ===== (this removes the hardcoded credentials) Then, at the end of the file, delete the ===== line Delete the >>>>>> feature-branch line and save the file.

Once the merge conflict was resolved, I verified by opening the config.py file, secondly, there was no push error by GitHub.



```
nextwork-security-secretsmanager — vi • g
pick 6c3a9ca # first commit
pick 7e6497c # Delete .DS_Store
pick 8c63f73 # Update config.py
pick 201e292 # Update config.py
pick 9186d38 # Update config.py
pick 5e49bc2 # Updated config.py with hardcoded credentials
pick b0a695e # Updated config.py with Secrets Manager credentials

# Rebase b0a695e onto bc91e89 (7 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
#     specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

